

A Theory of Programming

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Introduction

This lecture proposes a stepwise introduction to a foundation of programming and the analyses to write reliable and efficient software.

There are many different frameworks. We choose here a formal logic where relations and functions are the main building blocks. This entails that the meaning of **functional programs** can often be made very precise. (We will cover imperative programs later.)

It is a good framework to get a sense of **static analyses**, and, as such, it stands a prominent place in the field of formal methods and **compiler construction**.

Introduction

In a later lecture, we will learn a specific programming language called **OCaml**, enabling you to put in practice what you have learnt.

Most textbooks and courses on OCaml propose a traditional approach: basic concepts, data structures, algorithms.

By contrast, this lecture is not specific to OCaml and is theoretical.

String rewriting

Let us consider a string of white and black beads, like $\circ \bullet \bullet \bullet \circ \circ \bullet$, and a one-player game whose unique move consists in replacing two adjacent beads with only one according to the rules

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \bullet \xrightarrow{\beta} \bullet \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

The rules α , β and γ make up a simple **string-rewriting system**.

Rules α and β can be conceived as:

'A black bead absorbs the white bead next to it.'

The goal of this game is to end up with as few beads as possible, so our example may lead to the **rewrites**

$$\circ \bullet \bullet \boxed{\bullet \circ} \circ \bullet \xrightarrow{\alpha} \circ \bullet \bullet \boxed{\bullet \circ} \bullet \xrightarrow{\alpha} \boxed{\circ \bullet} \bullet \bullet \bullet \xrightarrow{\beta} \bullet \bullet \boxed{\bullet \bullet} \xrightarrow{\gamma} \bullet \boxed{\bullet \circ} \xrightarrow{\alpha} \boxed{\bullet \bullet} \xrightarrow{\gamma} \circ$$

where the part of the string to be rewritten next is framed.

Normal forms and termination

Other compositions of the rules lead to the same result $\circ$ as well. Some others bring all-white or all-black strings, like $\circ\circ$ and $\bullet$.

Strings that can not be further rewritten, or **reduced**, are called **normal forms**. We may wonder whether all strings have a normal form, and, if so, if it is unique and, furthermore, if it is either all-white or all-black.

First, let us note that the system is **terminating**, that is, there is no infinite chain of rewrites, because the number of beads strictly decreases in all the rules.

This is not a necessary condition in general, for instance, $\circ\bullet \xrightarrow{\beta} \bullet\circ\circ$ would preserve termination because the composition $\beta\alpha\alpha$ would be equivalent to the original rule β .

In particular, this means that any string has a normal form.

Invariants

Second, notice how the parity of the number of black beads is **invariant** through each rule and how there is no rewrite rule for two adjacent white beads.

Therefore, if there are $2p$ initial black beads, then composing rules α and β lead to an all-black string, which can be reduced by applying rule γ to contiguous pairs of beads into an all-white string made of p beads.

Otherwise, the same all-black string can be reduced by applying alternatively γ and β on the left end or γ and α on the right end, yielding $\circ$.

Similarly, if there is an initial odd number of black beads, we always end up with one black bead.

Confluence

Consequently, normal forms are not unique. A simple example is

$$\circ\circ \xleftarrow{\gamma} \bullet\bullet\circ \xrightarrow{\alpha} \bullet\bullet \xrightarrow{\gamma} \circ$$

Systems where normal forms are unique are **confluent**.

If we think of a rewrite step as an elementary computation, it becomes clear that confluence is a desirable property for carrying out computations, as we would expect, for instance, an arithmetic expression to have the same value no matter the order of the small steps.

A systemic order of rule application is called a **reduction strategy**.

Completing a system

We can obtain confluence by adding the rule δ :

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \bullet \xrightarrow{\beta} \bullet \qquad \bullet \bullet \xrightarrow{\gamma} \circ \qquad \circ \circ \xrightarrow{\delta} \circ$$

The result of the game is now always one bead whose colour depends on the original parity of the black beads as before.

To see why, let us consider first that two non-overlapping parts of a string can be rewritten in parallel, so they are not a concern.

The interesting cases occur when two applications of rules (maybe of the same rule) lead to different strings because they do overlap. For instance,

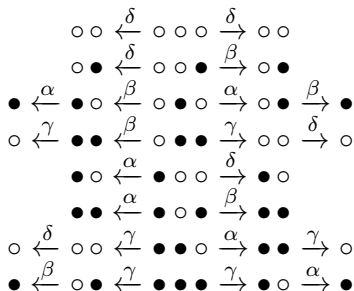
$$\circ \circ \xleftarrow{\gamma} \bullet \bullet \circ \xrightarrow{\alpha} \bullet \bullet$$

The important point is that $\circ \circ$ and $\bullet \bullet$ can be rewritten into $\circ$ at the next step by δ and γ , respectively.

Critical pairs

In general, what matters is that all pairs of strings resulting from the application of overlapping rules, called **critical pairs**, can be rewritten to the same string: they are **joinable**.

In our example, all interactions occur on substrings made of three beads, so we must examine eight cases:



In all the cases, the divergences are joinable in one step at most.

Local confluence

If all critical pairs are joinable after a divergence in one ore more rewrites, the system is **locally confluent**.

Moreover, if the system is also terminating, then **every string has exactly one normal form**.

Sometimes, as seen previously, a non-confluent system can be completed to become confluent, via a **completion procedure**.

In the case of terminating systems, one of such semi-decision procedures is the **Knuth-Bendix algorithm**.

Ground and linear rewrite systems

The system we defined is **ground**, that is, it involves no variables.

These allow a finite system to denote an infinite number of ground rules if variables range over infinite sets, or simply, they reduce the size of rewrite systems.

For instance, our continued example is equivalent to

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \ x \xrightarrow{\beta+\delta} x \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

If we accept multiple occurrences of a variable on the left-hand side of a rule (that is, not left-linear), we can further decrease the size of the system:

$$x \ x \xrightarrow{\gamma+\delta} \circ \qquad x \ y \xrightarrow{\alpha+\beta} \bullet$$

Ordered rewrite systems

Notice that there is now an implicit order over the rules: the rule $\gamma + \delta$ must be examined first for a **match** with a part of the current string, because it is included in the second (set $x = y$ in $\alpha + \beta$).

Non-linear systems are equational theories due the implicit equality on substructures, which is complex: we will limit ourselves to **linear systems**.

In general, the order in which the rules are written on the page is their **logical order**.

Moreover, let us remark that the system does not specify that x must either be $(\circ)$ or $(\bullet)$: this should be stated nevertheless somewhere else.

Generally speaking, this means that the **type** of the variables has to be defined elsewhere or inferred from the variable uses.

A simple calculator

Perhaps, after playing with beads, the simplest thing is an integer calculator, that is, integers and the four basic arithmetic operators.

An **expression** may be built using these operations, integers and parentheses. For example $(2 + 3) \times (4 + 5)$ is an expression.

A **value** is an integer, therefore it is a special case of expression.

What a calculator does is to match an input expression with a value, the result. For example, the above expression evaluates in 45.

This process is called **evaluation**.

Evaluation as rewriting expressions / Reductions

The evaluation of arithmetic expressions can be defined as a series of transformations leading to a value (the result). These transformations are called **rewrites** and a series of rewrites is a **reduction** or, more generally, a **computation**.

As we learnt in school long time ago, the way to perform these rewrites is to select a part of the current expression consisting of an operator and its arguments *as values*, then to replace this sub-expression by the operation result.

In the following reduction, let us underline the sub-expression to be rewritten and print in bold the result of each step:

$$(2 + 3) \times (\underline{4 + 5}) \rightarrow (\underline{2 + 3}) \times \mathbf{9} \rightarrow \mathbf{5} \times \underline{9} \rightarrow \mathbf{45}$$

Strategy of evaluation and rewrite systems

Note that there are different ways to choose the sub-expression. For example:

$$(\underline{2+3}) \times (4+5) \rightarrow \mathbf{5} \times (\underline{4+5}) \rightarrow \underline{5 \times 9} \rightarrow \mathbf{45}$$

It is usual to write, as a summary: $(2+3) \times (4+5) \xrightarrow{*} 45$ or $(2+3) \times (4+5) = 45$.

So, there are two concepts involved here:

1. the selection of a sub-expression made of an operator and its arguments as values,
2. the rewriting of such sub-expression into a value.

They are defined by a **rewrite system**.

Rewrite rules

A rewrite system is a set of **rewrite rules**.

Rewriting rules specify how to perform a rewrite.

For example, let us assume that we have an infinite set of rewriting rules, that is. an infinite rewrite system, like:

$$0 + 0 \rightarrow 0$$

$$0 + 1 \rightarrow 1$$

$$\dots \rightarrow \dots$$

$$124 - 57 \rightarrow 67$$

$$\dots \rightarrow \dots$$

Conditional rewrite rules and meta-variables

This rewrite system is not enough for the kind of expressions we want to evaluate, where it is possible to combine operations, like $(2 + 3) \times (4 + 5)$.

So let us extend our system with rules that specify the sub-expressions we can rewrite.

Consider the multiplication:

$$e_1 \times e_2 \rightarrow e'_1 \times e_2 \quad \text{if } e_1 \rightarrow e'_1$$

where e_1 , e'_1 and e_2 denote any expression and are called **meta-variables**.

Instance of a rewrite rule

An **instance** of this rule is obtained when the meta-variables are replaced by expressions.

For example, $(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)$ is an instance since $2 + 3 \rightarrow 5$. To understand why, replace e_1 by $2 + 3$, e'_1 by 5 and e_2 by $4 + 5$.

Let us rewrite $((2 + 3) \times 6) \times (4 + 5)$. First, we see that the previous rule has the required shape, i.e., the left side of the arrow is $e_1 \times e_2$ and if $e_1 = (2 + 3) \times 6$ and $e_2 = 4 + 5$ then $e_1 \times e_2$ is exactly the expression we wish to rewrite. So, we replace e_1 and e_2 by their associated expressions in the rule and we get an instance of it:

$$((2 + 3) \times 6) \times (4 + 5) \rightarrow e'_1 \times (4 + 5) \quad \text{if } (2 + 3) \times 6 \rightarrow e'_1$$

Therefore, we now know that we have to rewrite $(2 + 3) \times 6$.

Instance of a rewrite rule

We need another instance of the same rule. This time, let $e_1 = 2 + 3$ and $e_2 = 6$. We get

$$(2 + 3) \times 6 \rightarrow e'_1 \times 6 \quad \text{if } 2 + 3 \rightarrow e'_1$$

It is important to understand that this latter e'_1 is not the same as the former: they belong to two different instances of the same rule. The latter e'_1 is actually easy to find, since we have all the rules for basic cases as $2 + 3 \rightarrow 5$. Hence, $e'_1 = 5$ and the second instance is now complete:

$$(2 + 3) \times 6 \rightarrow 5 \times 6 \quad \text{if } 2 + 3 \rightarrow 5$$

Going backwards, this implies that the former e'_1 is 5×6 . So, the first instance is

$$((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5) \quad \text{if } 2 + 3 \rightarrow 5$$

Instance of a rewrite rule

Our rule for multiplication, $e_1 \times e_2 \rightarrow e'_1 \times e_2$, actually depends on another, $e_1 \rightarrow e'_1$. It is usual to write them vertically:

$$\frac{e_1 \rightarrow e'_1}{e_1 \times e_2 \rightarrow e'_1 \times e_2} \text{MULT}_1$$

This is called an **inference rule**. Note that it has a name, MULT_1 . The rewrite rule on the top, acting as a condition for the rule on the bottom, is called a **premise**. The rule at the bottom is called a **conclusion**.

A **substitution** is a mapping from meta-variables to expressions which applies to an inference rule or rewrite rule. For example, let σ be the following substitution: $\{e_1 \mapsto (2 + 3) \times 6; e_2 \mapsto 4 + 5\}$. This notation is equivalent to $\sigma(e_1) = (2 + 3) \times 6$ and $\sigma(e_2) = 4 + 5$.

Instance of a rewrite rule

Now we can make a clear definition of the notion of ‘instance of a rule’:

An instance of a rule is a rule in which some meta-variables e_i are replaced by expressions $\sigma(e_i)$.

For example, here is the instance of MULT_1 using previously defined substitution σ , and noted $\sigma(\text{MULT}_1)$:

$$\frac{(2 + 3) \times 6 \rightarrow e'_1}{((2 + 3) \times 6) \times (4 + 5) \rightarrow e'_1 \times (4 + 5)} \sigma(\text{MULT}_1)$$

Instance of a rewrite rule

Now how do we find (a mapping for) e'_1 ?

We define another substitution $\sigma' = \{e_1 \mapsto 2 + 3; e_2 \mapsto 6\}$ and apply it:

$$\frac{2 + 3 \rightarrow e'_1}{(2 + 3) \times 6 \rightarrow e'_1 \times 6} \sigma'(\text{MULT}_1)$$

Note that the meta-variable e'_1 in $\sigma'(\text{MULT}_1)$ is not the same as in $\sigma(\text{MULT}_1)$: they should have a different mapping.

We have a basic rewrite rule $2 + 3 \rightarrow 5$, thus we extend σ' with $e'_1 \mapsto 5$ and get

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times 6 \rightarrow 5 \times 6} \sigma'(\text{MULT}_1)$$

Instance of a rewrite rule

Therefore we can extend σ with $e'_1 \mapsto 5 \times 6$ and we get

$$\frac{(2 + 3) \times 6 \rightarrow 5 \times 6}{((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5)} \sigma(\text{MULT}_1)$$

We can summarise this rewrite using the two instances of MULT_1 as

$$\frac{\frac{2 + 3 \rightarrow 5}{(2 + 3) \times 6 \rightarrow 5 \times 6} \sigma'(\text{MULT}_1)}{((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5)} \sigma(\text{MULT}_1)$$

Instance of a rewrite rule

But what about the rewrite of an expression like $7 \times (4 + 5)$?

Let us define another substitution on MULT_1 :

$$\sigma' = \{e_1 \mapsto 7; e_2 \mapsto 4 + 5\}.$$

The corresponding instance is

$$\frac{7 \rightarrow e'_1}{7 \times (4 + 5) \rightarrow e'_1 \times (4 + 5)} \sigma'(\text{MULT}_1)$$

But there is no rule of shape ' $v \rightarrow \dots$ ', where v is a *value* (because a value is, by definition, an expression completely reduced).

Completing the multiplication rule

This means that we need another rule for the evaluation of the *right* argument of $(\times)$:

$$\frac{e_2 \rightarrow e'_2}{e_1 \times e_2 \rightarrow e_1 \times e'_2} \text{MULT}_2$$

This inference rule says that we can rewrite a product by rewriting its right argument.

For example, here is the instance $\sigma'(\text{MULT}_2)$:

$$\frac{4 + 5 \rightarrow 9}{7 \times (4 + 5) \rightarrow 7 \times 9} \sigma'(\text{MULT}_2)$$

Strategy

But now, something interesting happens.

Consider again $(2 + 3) \times (4 + 5)$: both MULT_1 and MULT_2 can be instantiated by substitution $\sigma'' = \{e_1 \mapsto 2 + 3; e_2 \mapsto 4 + 5\}$:

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)} \sigma''(\text{MULT}_1)$$

$$\frac{4 + 5 \rightarrow 9}{(2 + 3) \times (4 + 5) \rightarrow (2 + 3) \times 9} \sigma''(\text{MULT}_2)$$

Strategy

This means that *our system does not specify the order of evaluation of the arguments to $(\times)$* .

In other words, our rewrite system does not select only one sub-expression to rewrite at each step.

We need a **strategy** to complete it.

Finiteness and soundness of reductions

At this point, it is clear that these reductions satisfy the following properties:

- **Finiteness.** All the reductions are finite, i.e., all the computations terminate.
- **Soundness.** All the reductions of an expression end with the same value, if any. (Think confluence.)

About the second point, note indeed that *some expressions have no value*, as

$$(2 + 3)/(4 - 4) \rightarrow 5/(\underline{4 - 4}) \rightarrow 5/0 \nrightarrow$$

The reduction is finite but does not end with a value: this situation corresponds to an **error**, usually called **run-time error** in programming languages. Expression $5/0$ is **stuck**.

Another model of run-time errors

We can modify the calculator so that, in case of error (here, the only one is the division by zero), the result of a rewrite is a special value, called NaN (*Not a Number*). Then we add the following rules:

$$e/0 \rightarrow \text{NaN} \quad \text{DIVZERO} \qquad \text{NaN}/e \rightarrow \text{NaN} \quad \text{DIV-ERR}_1$$

$$e/\text{NaN} \rightarrow \text{NaN} \quad \text{DIV-ERR}_2 \qquad \text{NaN} \times e \rightarrow \text{NaN} \quad \text{MULT-ERR}_1$$

$$e \times \text{NaN} \rightarrow \text{NaN} \quad \text{MULT-ERR}_2 \qquad \text{NaN} + e \rightarrow \text{NaN} \quad \text{ADD-ERR}_1$$

$$e + \text{NaN} \rightarrow \text{NaN} \quad \text{ADD-ERR}_2 \qquad \text{NaN} - e \rightarrow \text{NaN} \quad \text{SUB-ERR}_1$$

$$e - \text{NaN} \rightarrow \text{NaN} \quad \text{SUB-ERR}_2$$

Another model of run-time errors

The rationale of this system is that once an error occurs, no other computations (leading to integer values) are required and the reduction can end as soon as possible.

In this case:

$$(2 + 3)/(4 - 4) \rightarrow 5/(4 - 4) \rightarrow 5/0 \rightarrow \text{NaN}$$

or, even the shortest possible:

$$(2 + 3)/(4 - 4) \rightarrow (2 + 3)/0 \rightarrow \text{NaN}$$

Rephrasing soundness

The interesting phenomenon here is that, now, any reduction ends in a value (since errors are values). In the previous rewrite system, any reduction ends either in a value or a **stuck expression**, i.e., an expression that cannot be rewritten further.

With the new system, that explicitly specifies the reduction of the NaN error, the soundness property can be rephrased as

All the reductions of an expression end with the same value.

Obviously, the drawback of the new system is that it requires a lot of additional rules, and, as the system is augmented with new rules, new rules for the errors have to be added... or are forgotten by mistake.

Specifying the strategy

It is possible to force an order of evaluation on the arguments of $(\times)$. Assume we want to reduce the left argument first.

The idea is to change MULT_2 so that instead of e_1 , which stands for any expression, we specify a value v :

$$\frac{e_2 \rightarrow e'_2}{v \times e_2 \rightarrow v \times e'_2} \text{NEW-MULT}_2$$

This way, it is not possible anymore to instantiate this rule with expression $(2 + 3) \times (4 + 5)$ because $2 + 3$ is not a value.

Specifying the strategy

The only possible reduction is:

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)} \sigma_1(\text{MULT}_1)$$

$$\frac{4 + 5 \rightarrow 9}{5 \times (4 + 5) \rightarrow 5 \times 9} \sigma_2(\text{NEW-MULT}_2)$$

where $\sigma_1 = \{e_1 \mapsto 2 + 3; e_2 \mapsto 4 + 5\}$ and $\sigma_2 = \{v \mapsto 5; e_2 \mapsto 4 + 5\}$.

Determinism

We have specified the strategy **in** the rewrite system.

Also, with NEW-MULT_2 , given an expression, there is only one rule that can be applied (or none): this is sometimes called **determinism**.

Determinism. If $e_1 \rightarrow e'_1$ and $e_2 \rightarrow e'_2$ then $e'_1 = e'_2$.

This is a stronger property than soundness, i.e., it implies soundness. Indeed, determinism implies that there is only one reduction from an expression to its value (if any).

Pragmatics of programming languages

In many programming languages, the order of evaluation of operator arguments is not specified, which usually means that there is a sentence in the official document which defines the language like: 'The order of evaluation of the arguments of operators and functions is not specified.'

Formally, this means that the underlying rewrite system is **non-deterministic**.

One notable exception is Java, which specifies that the arguments are always evaluated from left to right, i.e., following the order of writing.

Of course, if an order is specified, as in Java, the compilers of these languages must implement it. Otherwise one is chosen arbitrarily and may change from one release to another.

Rewrite systems as models of languages

By studying rewrite systems, we can learn the basic concepts involved in programming languages, because rewrite systems are extremely simple.

There are some languages, like Maude, whose programs actually are rewrite systems.

Some compilers transform the input program (called **source**) and output a program, called **byte-code**, which can be reduced by a rewrite system.

Rewrite systems as models of languages

This is the case of Java. This is why, in general, we need a **Java virtual machine** (JVM) to run the program: this JVM performs the reductions according to a rewrite system.

Even if there is no byte-code, we always can interpret what the program does by studying a rewrite system which models it.

Sequentiality and parallelism

Some programming languages, like Ada, allow the programmer to specify that different parts of its program can actually be executed separately: this is called, in general, **parallelism**. This parallelism is possible only if the underlying rewrite system is non-deterministic.

Indeed, if all the strategies lead to the same value (soundness property), it is possible to reduce different parts of an expression in parallel if the rewrite system allows it.

Sequentiality and parallelism

For example, the first argument of $(\times)$ can be computed *at the same time* as its second argument. In the case of $(2 + 3) \times (4 + 5)$, this means that we can reduce in parallel $2 + 3$, with MULT_1 , and $4 + 5$, with MULT_2 , then 5×9 is finally rewritten in 45 (after **synchronisation** of the two reductions).

Order of evaluation in C

In case of C, the order of evaluation arguments of arithmetic operators and functions is not specified, it actually depends on the compiler and its version. Therefore it would be dangerous to rely on the order of evaluation in C.

Why is the order left unspecified? Simply because of possible **side-effects**. Imagine the following situation:

```
int n = 0;
int f () { n++; return n; };
int g () { n = 2; return n; };
int main () { return f() + g() + n; }
```

What is the result?

Boolean expressions

Let $\wedge$ represent the conjunction ('and'), $\vee$ the disjunction ('or'), $\neg$ the negation ('not'), and let the two values true and false.

The definition of the conjunction of values is very simple:

$$\text{true} \wedge \text{true} \rightarrow \text{true}$$

$$\text{true} \wedge \text{false} \rightarrow \text{false}$$

$$\text{false} \wedge \text{true} \rightarrow \text{false}$$

$$\text{false} \wedge \text{false} \rightarrow \text{false}$$

Boolean expressions made simpler

This system can even be made simpler, by means of meta-variables:

$$\text{true} \wedge \text{true} \rightarrow \text{true} \quad \text{TRUE}$$

$$\text{false} \wedge v \rightarrow \text{false} \quad \text{FALSE}_1$$

$$v \wedge \text{false} \rightarrow \text{false} \quad \text{FALSE}_2$$

where v denotes any value, i.e., either true or false.

Boolean expressions

So, if we define the rewrite of boolean expressions (not just the values true and false) as

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge_1 \qquad \frac{e_2 \rightarrow e'_2}{e_1 \wedge e_2 \rightarrow e_1 \wedge e'_2} \wedge_2$$

then, for example,

$$\frac{\text{false} \wedge \text{true} \rightarrow \text{false} \quad \text{FALSE}_1}{(\text{false} \wedge \text{true}) \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \wedge (\text{true} \wedge \text{true})} \sigma_1(\wedge_1)$$

where $\sigma_1 = \{e_1 \mapsto \text{false} \wedge \text{true}; e_2 \mapsto \text{true} \wedge \text{true}; e'_1 \mapsto \text{false}\}$.

Boolean expressions

Then

$$\frac{\text{true} \wedge \text{true} \rightarrow \text{true} \quad \text{TRUE}}{\text{false} \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \wedge \text{true}} \sigma_2(\wedge_2)$$

where $\sigma_2 = \{e_1 \mapsto \text{false}; e_2 \mapsto \text{true} \wedge \text{true}; e'_1 \mapsto \text{true}\}$.

and, finally, we can instantiate FALSE_1 with $\sigma_3 = \{v \mapsto \text{true}\}$:

$$\text{false} \wedge \text{true} \rightarrow \text{false} \quad \sigma_3(\text{FALSE}_1)$$

In this example, we computed both arguments of $\wedge$ to reach the value.

Boolean expressions

But the inspection of the basic rules shows that if one of the arguments of $\wedge$ reduces to false, then there is no need to reduce the other argument: the rewrite will always be false. Therefore, it is more efficient to extend the rules on values to the expressions:

$$\text{false} \wedge e \rightarrow \text{false} \quad \text{FALSE}_1 \qquad e \wedge \text{false} \rightarrow \text{false} \quad \text{FALSE}_2$$

$$\text{true} \wedge e \rightarrow e \quad \text{TRUE}_1 \qquad e \wedge \text{true} \rightarrow e \quad \text{TRUE}_2$$

where e represents any boolean expression. Then, now

$$\text{false} \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \quad \sigma_3(\text{FALSE}_1)$$

where $\sigma_3 = \{e \mapsto \text{true} \wedge \text{true}\}$.

Boolean expressions

Try to reduce

$$(\text{true} \wedge (\text{false} \wedge \text{true})) \wedge (\text{true} \wedge (\text{true} \wedge \text{true}))$$

Note that this rewrite system is not deterministic. If we enforce that the left argument of $\wedge$ is completely reduced first, then we can check if it is true or false: in the latter case, we do not need to reduce the right argument.

Boolean expressions

This is easy to specify:

$$\text{true} \wedge e \rightarrow e \quad \wedge_{\text{TRUE}} \qquad \text{false} \wedge e \rightarrow \text{false} \quad \wedge_{\text{FALSE}}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge$$

Many programming languages, as C, use this strategy. It allows the programmers to write handy code like

```
while (index < max && tab[index] > 0) ...
```

Boolean expressions

Note that this strategy (left argument before the right one) is not optimal in general: the first argument may reduce to true. *Finding an optimal strategy is not always possible.* For example, we could add another rule that shortens sometimes the reductions:

$$e \wedge e \rightarrow e \wedge_{\text{REFL}}$$

This rule explicitly specifies the **reflexivity** of the conjunction. But the system becomes non-deterministic because there is now an overlap between rule $\wedge_{\text{REFL}}$ and both $\wedge_{\text{TRUE}}$ and $\wedge_{\text{FALSE}}$ (consider for example $\text{true} \wedge \text{false}$).

Determinism versus non-determinism

Let us summarise the concepts of determinism and non-determinism.

- Determinism means that, for any expression, there is only one rule to rewrite it. This implies that, every time a program is run with the same input, the output will be the same. This is a useful property, called soundness, in particular for debugging.
- Non-determinism means that, for any expression, there may be several rules to rewrite it. Moreover,
 - if the rewrite system is sound, the output will always be the same for any run on the same input, even if the trace may be different each time, which makes the debugging more difficult, but enable parallel reductions.
 - the rewrite system may be unsound.

Determinism versus non-determinism

Some programming languages have features that may be both non-deterministic and unsound.

Perhaps the most infamous case is in C, where variables do not need to be initialised at their declaration:

```
int a;
```

If the programmer forgets to give a value to the variable, any access to it is unspecified.

Practically, this means that whatever is located in the memory at the location of the variable will be used at each read access. Of course, this value may change from one run of the program to another, even on the same input.

Commutativity and its consequences

We may try another rewrite system for $\wedge$:

$$\text{true} \wedge e \rightarrow e \quad \text{TRUE}$$

$$\text{false} \wedge e \rightarrow e \quad \text{FALSE}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge$$

$$e_1 \wedge e_2 \rightarrow e_2 \wedge e_1 \quad \text{COMM}$$

While this system is in theory fine, there are two practical problems:

1. it is **not deterministic** due to COMM that overlaps with all the other rules;
2. it may **not terminate** due to rule COMM:

$$\begin{aligned} \text{true} \wedge \text{false} &\xrightarrow{\text{COMM}} \text{false} \wedge \text{true} \xrightarrow{\text{COMM}} \text{true} \wedge \text{false} \\ &\xrightarrow{\text{COMM}} \text{false} \wedge \text{true} \rightarrow \dots \end{aligned}$$

Non-termination

Now some reductions may not be finite, which models **non-termination**.

This particular kind of non-termination, due to commutativity, is the model in programming languages of so-called '**infinite loops**' that do not allocate memory, i.e., that do not require additional memory at each loop step. Equivalently, it corresponds to a *recursive call* like

```
bool and(bool e1, bool e2) {  
    ... return and(e2, e1); ... }
```

Anyway, in the presence of infinite reductions, the soundness property must be weakened as follows:

Weak soundness. *All finite reductions of an expression end with the same value, if any.*

Non-termination

Let us define the negation:

$$\neg \text{true} \rightarrow \text{false}$$

$$\neg \text{false} \rightarrow \text{true}$$

It is also well known that the negation satisfies $e = \neg\neg e$. So we could imagine another rewrite rule

$$\neg\neg e \rightarrow e$$

This is fine, this rule simplifies double negations. But what if we had the converse rule?

$$e \rightarrow \neg\neg e \quad \text{where } e \text{ is not a value.}$$

This allows infinite reductions: $e \rightarrow \neg\neg e \rightarrow e \rightarrow \neg\neg e \rightarrow \dots$

Summary

There are three kinds of reductions:

1. Finite and ending in a value

$$e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

2. Finite but ending in a stuck expression

$$e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow e_n \nrightarrow$$

3. Infinite

$$e_1 \rightarrow e_2 \rightarrow \cdots$$

Conditional expressions

At this point, boolean expressions can be

- **true** (a value)
- **false** (a value)
- $e_1 \wedge e_2$
- $e_1 \vee e_2$
- $\neg e_1$

where e_1 and e_2 are boolean expressions. Let us extend our boolean expressions with a **conditional expression**:

- **if** e_1 **then** e_2 **else** e_3

Conditional expressions

It is easy to extend the rewrite system to cope with conditionals:

$$\text{if true then } e_2 \text{ else } e_3 \rightarrow e_2 \quad \text{IF-TRUE}$$

$$\text{if false then } e_2 \text{ else } e_3 \rightarrow e_3 \quad \text{IF-FALSE}$$

$$\frac{e_1 \rightarrow e'_1}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rightarrow \text{if } e'_1 \text{ then } e_2 \text{ else } e_3} \quad \text{IF}$$

This means that the boolean expression e_1 must be completely reduced *before* the others. This is the usual way in programming languages, the rationale being to avoid unnecessary computations (evaluating e_2 and e_3 is not necessary).

Term rewriting

More general are the **term-rewriting systems**, where a **term** is a mathematical object built on tuples, integers and variables.

Let us consider the following totally ordered system:

$$\begin{aligned}(0, m) &\xrightarrow{1} m \\(n, m) &\xrightarrow{2} (n-1, n \times m) \\n &\xrightarrow{3} (n, 1)\end{aligned}$$

Arithmetic operators $(-)$ and $(\cdot)$ are defined out of the system and m and n are variables denoting natural numbers.

Would the rules not be ordered as they are actually laid out, the second rule would match any pair. Instead, it can be assumed that $n \neq 0$ when matching with it.

The last rule is to be tried last, as it matches all terms.

Transitive closures

We can easily see that all the compositions of rewrites starting with a natural number n end with the factorial of n :

$$n \xrightarrow{3} (n, 1) \xrightarrow{2} \dots \xrightarrow{2} (0, n!) \xrightarrow{1} n!, \quad \text{for } n \in \mathbb{N}.$$

Let us note $(\xrightarrow{n})$ the composition of $(\rightarrow)$ repeated $n - 1$ times:

$$\begin{aligned} (\xrightarrow{1}) &:= (\rightarrow) \\ (\xrightarrow{n+1}) &:= (\rightarrow) \circ (\xrightarrow{n}), \quad \text{with } n > 0. \end{aligned}$$

The **transitive closure** of $(\rightarrow)$ is defined as

$$(\rightarrow^*) := \bigcup_{i > 0} (\xrightarrow{i}).$$

The factorial function coincides with the transitive closure of $(\rightarrow)$: $n \rightarrow^* n!$. Let $(\xrightarrow{*})$ be the reflexive-transitive closure of $(\rightarrow)$, that is,

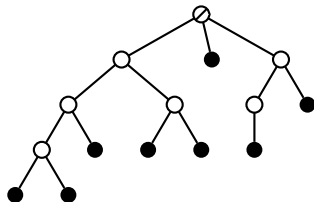
$$(\xrightarrow{*}) := (=) \cup (\rightarrow^*).$$

Functions

- A confluent system defines a **partial function**.
- If the system is also terminating, we get a **function**.
- We can name functions: for example, $c(1, d(n))$ is a term constructed with **function names** c and d , as well as variable n .
- A tuple tagged with a function name, like $f(x, y)$, is called a **function call**.
- The components of the tuples are then called **arguments**, for example $d(n)$ is the second argument of the call $c(1, d(n))$.
- It is possible for a function call to hold no arguments, like $d()$.
- We restrict the left-hand sides of rules to be function calls.

Trees

The topological understanding of a function call or a tuple is the finite **tree**. A tree is a hierarchical layout of information, for example:



The disks are called **nodes** and the segments which connect two nodes are called **edges**. The topmost node (with a diameter) is called the **root** and the bottommost ones (●) are called the **leaves**. All nodes except the leaves are seen to downwardly connect to some other nodes, called **children**. Upwardly, each node but the root is connected to another node, called its **parent**.

Trees

Trees are pervasive in informatics, especially in functional programming, but not only. Structured documents, like books, articles, and web pages, are composed of chapters, sections, paragraphs, figures, appendices, indices, etc.

The occurrences of these components are mutually constrained; for instance, it is understood that a section is part of a chapter and that appendices are located at the end of a document.

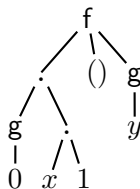
This hierarchical layout is meant to facilitate reading, and it supports the search for specific items of information.

When considering computer systems, these data must be uniformly encoded by means of a formal language.

Terms as trees

Trees can be used to depict terms as follows. A function call is a tree whose root is the function name and the children are the trees denoting the arguments.

A tuple can be considered as having an invisible function name represented by a node with a period (.) in the tree, in which case the components of the tuple are its children. For example,



is the tree corresponding to the term $f((g(0), (x, 1)), (), g(y))$.

Note that tuples, except the empty tuple, are not represented in the tree, since they encode the structure itself, which is already laid out. The number of arguments of a function is called **arity**.

Functional languages

- For programming, we only consider confluent systems because they define partial functions.
- To have only one reduction chain per term to the normal form, we may also enforce that rules are ordered.
- We also enforce that normal forms must be **values**, that is, terms without function calls or only with calls to functions not occurring on the left-hand sides.
- In other words, irreducible calls of defined functions are not allowed in values.
- Systems with these constraints make up a **functional language**.
- A program here is a term, and evaluating it means to rewrite the term
 1. to a value,
 2. or to end in error with a normal form that is not a value,
 3. or to never terminate.

Call-by-value

If a function call contains at least one other function call, there is a choice as to which one to reduce first.

One reduction strategy, named **call-by-value**, consists in always reducing the arguments before the call itself. Unfortunately, call-by-value enables otherwise terminating rewrites to not terminate. For instance, let us consider

$$f(x) \xrightarrow{\alpha} 0 \qquad g() \xrightarrow{\beta} g()$$

We have $f(g()) \xrightarrow{\alpha} 0$ but $f(g()) \xrightarrow{\beta} f(g()) \xrightarrow{\beta} \dots$

Despite the loss of expressiveness, we shall retain in this lecture the call by value, as it is the strategy of OCaml and it is easy to follow.

In general, the order in which the arguments of a function call are evaluated is not specified.

Call-by-value

Another reason to choose call-by-value is that it allows us to restrict the shape of the left-hand sides, called **patterns**, to one, outermost function call.

For instance, we can then disallow, for being useless, a rule like

$$\text{plus}(x, \text{plus}(y, z)) \rightarrow \text{plus}(\text{plus}(x, y), z).$$

because, following call-by-value, the argument $\text{plus}(y, z)$ must be evaluated before the outermost call, therefore it cannot be part of any pattern.

Termination

We do not require by construction that all functional systems terminate, although that is a desirable property, but we nevertheless speak of ‘functions’ instead of ‘partial functions’ (which would be technically correct).

If we enforce termination, we loose **Turing-completeness**: we cannot program all computable functions.

The termination of rewrite systems and, in particular, functional programs, is an **undecidable problem** in general.

Nevertheless, there exists a set of well-known **decision criteria**, which we will use when programming recursive functions in OCaml.

Higher-order programs, recursion, λ -calculus

Most functional languages allow **higher-order function** definitions, whereas standard term-rewriting systems do not. For example:

$$f(g, 0) \rightarrow 1 \quad f(g, n) \rightarrow n \times g(g, n - 1) \quad \text{fact}(n) \rightarrow f(f, n)$$

where $n \in \mathbb{N}$. Note that these two definitions are **not** recursive, yet the function `fact` is the factorial function. A function definition is **recursive** if the name of the definition occurs on at least one right-hand side.

An adequate theoretical framework to understand higher-order functions is **λ -calculus**. In fact, λ -calculus features prominently in the semantics of programming languages, even not functional ones.

For didactic purposes, we began with rewrite systems because they offer implicit pattern matching (that is, rule selection).

Stacks in a functional setting

We show how to express linear data structures in a purely functional language and how to compute with them. The simplest of those structures is the **stack**.

A stack can be thought of as a finite series of items that can only be accessed sequentially from one end, called the **top**, following the analogy with a stack of material objects.

Stacks in a functional setting

To define a stack in a functional language, one uses calls to undefined functions to model all the possible shapes a stack can take.

Here, a stack can be either empty or not.

Let $\text{nil}()$ be the irreducible call that denotes an empty stack (that is, the function 'nil' is undefined).

If not empty, the stack then must contain a topmost (first) item, and a **substack**, that is, the stack made of all the items but the first.

Let $\text{cons}(x, s)$ denote the non-empty stack with the item x on top and the **immediate substack** s .

Inductive definition of stacks

- The previous definition is intuitive, but not formal yet.
- Data structures are usually defined **inductively**.
- Let $\mathcal{T}$ be the set of all possible terms and $\mathcal{S} \subset \mathcal{T}$ be the set of all stacks.
- Formally, $\mathcal{S}$ can be defined by **induction** as the smallest set such that
 1. $\text{nil}() \in \mathcal{S}$;
 2. if $x \in \mathcal{T}$ and $s \in \mathcal{S}$, then $\text{cons}(x, s) \in \mathcal{S}$.
- We take the smallest set because we do not want in $\mathcal{S}$ terms that were not constructed by the inductive rules (this is called the **smallest fixed point** of the inductive relation).

Appending a stack

- For all $s, t \in \mathcal{S}$, let us consider the functional program

$$\begin{aligned}\text{cat}(\text{nil}(), t) &\xrightarrow{\alpha} t \\ \text{cat}(\text{cons}(x, s), t) &\xrightarrow{\beta} \text{cons}(x, \text{cat}(s, t))\end{aligned}$$

- This is a functional program suitable for call-by-value reduction strategy, because the function calls in the patterns (that is, left-hand sides) are irreducible.
- The right-hand sides are stacks, so a call to 'cat' is a stack.
- Rule α says that, if the first stack is empty, the value is the second.
- In rule β , the first stack is not empty, and its top item is the top of the final stack (x), the immediate substack (s) being used in a call to 'cat' with the second stack (t) unchanged.

Appending a stack

This function appends the second stack at the bottom of a **copy** of the first. (The function name 'cat' stands for 'catenation', and is slightly better than 'app').

We could also say that this function appends a copy of the first stack on top of the other. But this is not the same as pushing the first stack on top of the other, like

$$\text{cat}(s, t) \rightarrow \text{cons}(s, t) \quad \text{Wrong!}$$

This is **not** appending: here, the stack s becomes an item of the resulting stack, but we want only the **contents** of s to be pushed on top of t , not their container.

Analogy: You may think of an empty stack as a empty cardboard box, open topside, and every item pushed in it is like a book that you put on the topmost book, or at the bottom of the box if this is the first book.

Appending a stack

Let us set the abbreviations

- $[] := \text{nil}()$,
- $x :: s := \text{cons}(x, s)$,

after the convention of the programming language OCaml.

We can further abbreviate

- $x_1 :: x_2 :: \dots :: x_n :: s := \text{cons}(x_1, \text{cons}(x_2, \dots \text{cons}(x_n, s)))$
- $[x] := x :: []$
- $[x_1; x_2, \dots; x_n] := x_1 :: x_2 :: x_3 :: \dots :: x_n :: []$.

Our system now becomes a bit more legible:

$$\begin{aligned}\text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t)\end{aligned}$$

Pattern matching

- Let us compute $\text{cat}([1; 2], [3])$.
- We need to find the first rule that is matched by this call.
- Remember that the arguments must be values before you do that.
- Intuitively, this means that we compare the call with the patterns, that is, the left-hand sides of the rules, in order.
- Here, rule α is not matched because $[1; 2] \neq []$.
- Rule β is matched because $[1; 2] = 1 :: [2]$ and equals the sub-pattern $x :: s$ if we assign $x := 1$ and $s := [2]$. Also, trivially, $t := [3]$ works.
- After the rule β is selected, the assignments are used to replace the variables in the right-hand sides.
- The whole evaluation is

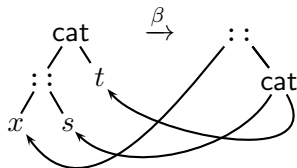
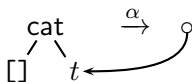
$$\text{cat}([1; 2], [3]) \xrightarrow{\beta} 1 :: \text{cat}([2], [3]) \xrightarrow{\beta} 1 :: 2 :: \text{cat}([], [3]) \xrightarrow{\alpha} [1; 2; 3].$$

Abstract Syntax Trees (ASTs)

- Depending on the context, we may use the arborescent depiction of terms to bring to the fore certain aspects of a computation.
- For example, it may be interesting to show how parts of the output (the right-hand side) are actually **shared** with the input (the left-hand side).
- In other words, how much (new) data is created by a given rule.
- This concept supposes that terms reside in some sort of space and that they can be referred to from different other terms.
- This abstract space serves as a model of an interpreter's memory, called the **heap**.

Data sharing

- Consider for instance



which is the same definition of 'cat' as given above:

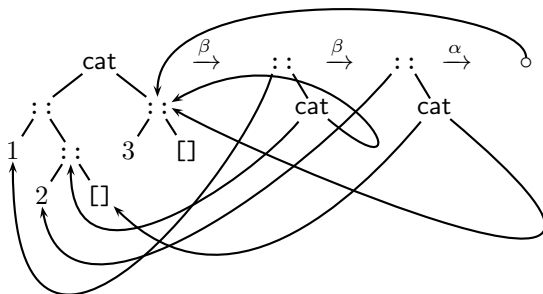
$$\text{cat}([], t) \xrightarrow{\alpha} t$$

$$\text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t)$$

- The arrows on certain edges denote data sharing.
- When trees are used to visualise terms, they are called **abstract syntax trees** (AST).
- When some trees share subtrees, the whole **forest** (set of trees) is called a **directed acyclic graph** (DAG).

Data sharing and evaluation

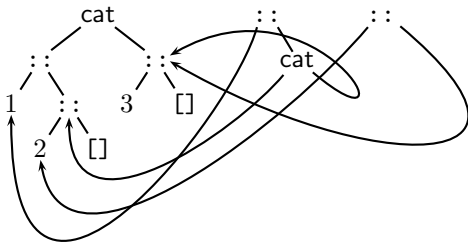
The evaluation of $\text{cat}([1; 2], [3])$ is:



Note that each right-hand side shows only the rewritten call, not the whole term.

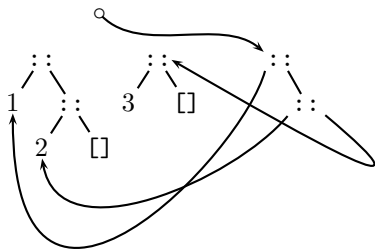
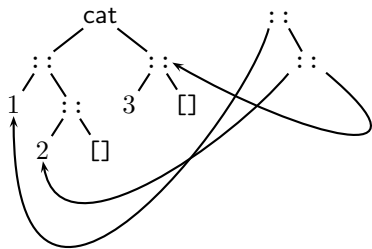
Data sharing and evaluation

Therefore, to reconstruct the value, we need to work leftwards and replace the rewritten call by the right-hand side:



We do not show the calls that have been replaced by their values.

Data sharing and evaluation



Call stack

We can see that the value of the second argument, [3], is shared with the value of the call, as well as the integers 1 and 2.

We saw how rewrites accumulated rightwards until the result had to be reconstructed by replacing leftwards the right-hand sides.

This dynamic is that of a stack, but a stack holding suspended function calls (that is, awaiting their values) and references to ASTs in the heap. This is the **call stack**.

The top of the stack (here, the rightmost abstract syntax tree) is the next term to be rewritten, unless the call on the left-hand side is the original call, in which case the top is its value.

Garbage collection

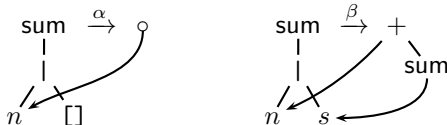
- In the abstract syntax trees (AST), the nodes with the calls that are replaced by their values have been immediately removed. In fact, they could have been removed later, in any order.
- The process which identifies and discards useless ASTs is called the **garbage collector** (GC).
- A term is useless if it cannot be accessed from anywhere in the call stack.
- Contrary to the heap, which can be thought of as an unordered set of trees (a forest of DAGs), the call stack is an ordered set that can be efficiently extended or reduced.
- The garbage collector can be conceived as a process interleaved with the process of evaluation.
- Like evaluators of functional programs, garbage collectors may also have different strategies (as to when dispose of useless data).

Tail call optimisation

Let us define a function that sums integers in a non-empty list:

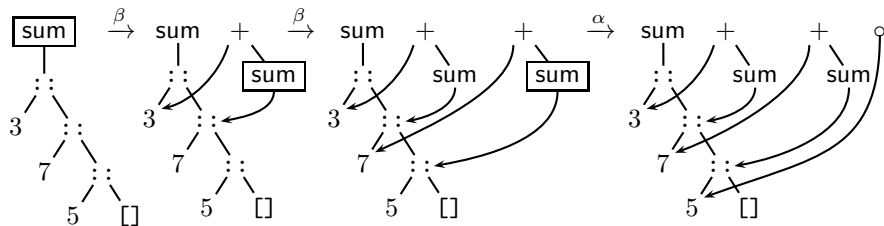
$$\text{sum}([n]) \xrightarrow{\alpha} n \qquad \text{sum}(n::s) \xrightarrow{\beta} n + \text{sum}(s)$$

The view of the system with maximum sharing is:



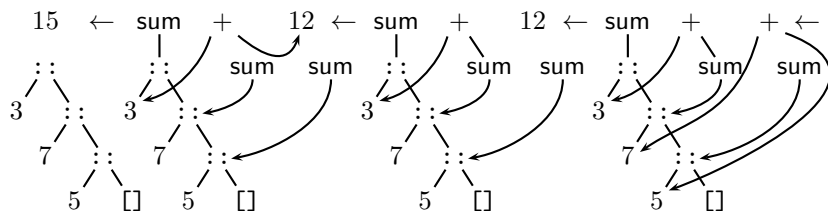
Tail call optimisation

The first phase of the evaluation of $\text{sum}([3; 7; 5])$ is:



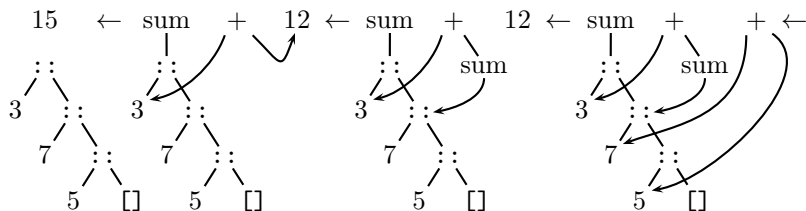
Tail call optimisation

The second phase (replacing the pending calls with their values) is



Tail call optimisation

The same, with a garbage collector that collects after each replacement:

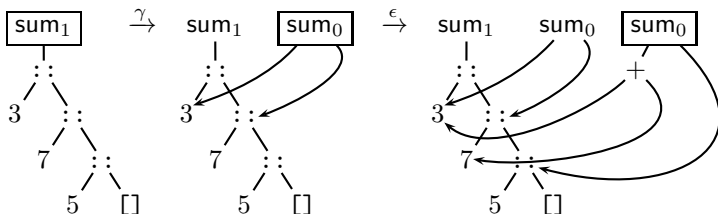


Tail call optimisation

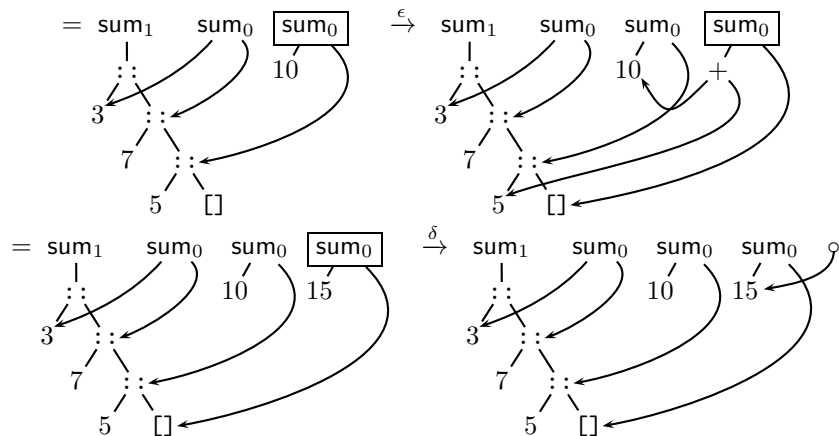
Let us consider an alternate version of sum:

$$\begin{aligned}\text{sum}_1(n :: t) &\xrightarrow{\gamma} \text{sum}_0(n, t). \\ \text{sum}_0(n, []) &\xrightarrow{\delta} n \\ \text{sum}_0(n, m :: s) &\xrightarrow{\epsilon} \text{sum}_0(n + m, s).\end{aligned}$$

For $\text{sum}_1([3; 7; 5])$ we have the evaluation



Tail call optimisation

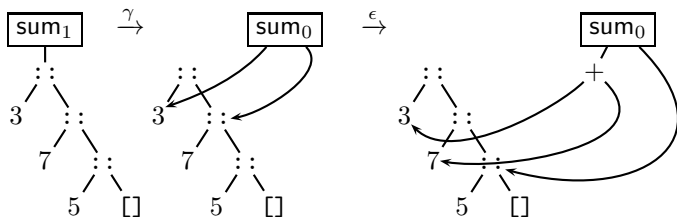


Tail call optimisation

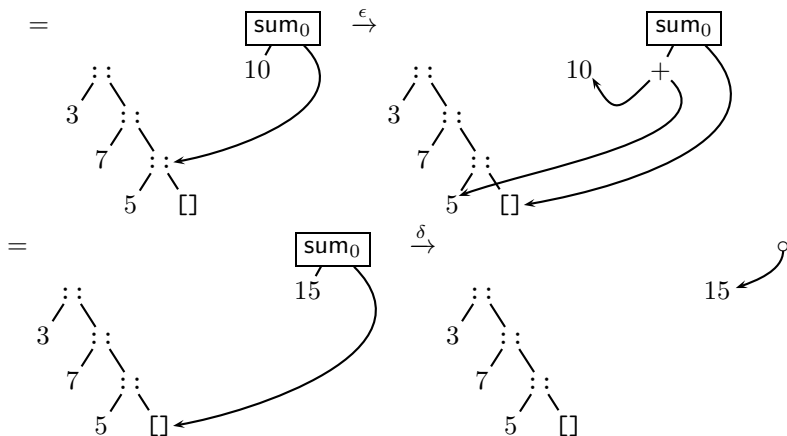
Notice that, in the case of sum_0 , we found the result (15) at the end of the first phase (of growing the call stack).

Therefore, there is no need for a second phase, which replaces the pending calls by their values: we can discard all those calls at once after the first phase.

Or, to see it even more clearly, let us rewrite the same first phase and assume that the collector reclaims immediately the useless pending calls:



Tail call optimisation



Tail call optimisation

As we see now, we can keep the size of the call stack constant, if we reclaim the top at each new rewrite.

This optimisation is called **tail call optimisation**.

Intuitively, a call in **tail position** is a call whose value is the value of the previously rewritten call.

For example, the call $f(x)$ in $1 + f(x)$ is not in tail position, because its value needs to be incremented.

Most compilers of functional languages detect and implement that optimisation that saves memory, including the C compiler of GCC.

Inductive proofs

Let us remark that $\text{cat}([1], [2, 3, 4]) \twoheadrightarrow [1, 2, 3, 4] \longleftarrow \text{cat}([1, 2], [3, 4])$.

It is enlightening to create *equivalence classes* of terms that are joinable. These classes then define an equivalence relationship ($\equiv$) as follows:

$a \equiv b$ if there exists a unique value v such that $a \xrightarrow{*} v$ and $b \xrightarrow{*} v$.

For instance, $\text{cat}([1, 2], [3, 4]) \equiv \text{cat}([1], [2, 3, 4])$.

The relation ($\equiv$) is indeed an equivalence because it is

- *reflexive*: $a \equiv a$;
- *symmetric*: if $a \equiv b$, then $b \equiv a$;
- *transitive*: if $a \equiv b$ and $b \equiv c$, then $a \equiv c$.

Inductive proofs

Of some interest are the following facts.

- If $f(x)$ and $f(y)$ have a value, then $x \equiv y$ implies $f(x) \equiv f(y)$.
- If $x_1 \leftarrow x_2 \rightarrow x_3 \equiv x_4 \rightarrow x_5 \leftarrow x_6$, then
 $x_1 \equiv x_2 \equiv x_3 \equiv x_4 \equiv x_5 \equiv x_6$.

If we want to prove equivalences with variables ranging over infinite sets, like $\text{cat}(s, \text{cat}(t, u)) \equiv \text{cat}(\text{cat}(s, t), u)$, we need some **induction principle**.

Well-founded induction

We define a **well-founded order** on a set A as being a binary relation ($\succ$) such that it is

- irreflexive: $\forall x \in A. \neg(x \succ x)$,
- transitive: $\forall x, y, z \in A. x \succ y \wedge y \succ z \Rightarrow x \succ z$,
- and does not have any **infinite descending chains**, to wit, there is no $a_0 \succ a_1 \succ a_2 \succ \dots$

The **well-founded induction principle** then states that, for any predicate $\mathbb{N}$ on a set A ,

$$\forall a \in A. \mathbb{N}(a) \text{ is implied by } \forall a. (\forall b. a \succ b \Rightarrow \mathbb{N}(b)) \Rightarrow \mathbb{N}(a).$$

When $A = \mathbb{N}$, this principle is called **mathematical (complete) induction**.

Well-founded induction

Structural induction is another particular case where $t \succ s$ holds if, and only if, s is a proper **subterm** of t , namely, the abstract syntax tree of s is included in the tree of t and $s \neq t$.

Sometimes, a restricted form of structural induction is enough. For instance, we can define $x :: s \succ s$, for any term x and any stack $s \in S$. Both x and s are **immediate subterms** of $x :: s$. (In other words, we dispensed with the transitivity axiom.) Because there are no infinite descending chains, any subset $B \subseteq A$ contains minimal elements $M \subseteq B$, that is, there is no $b \in B$ such that $a \succ b$, if $a \in M$. So

1. we have to prove $\mathcal{N}(a)$ for all $a \in M$, the so-called **basis**: here, only $\mathcal{N}([])$, as $[]$ is by construction the unique, minimal stacks, and,
2. assuming the so-called **induction hypothesis** $\mathcal{N}(s)$, we must prove $\mathcal{N}(x :: s)$ for all x .

Termination is undecidable

On pages 5 and 66, we introduced the property of a rewrite system to terminate on all inputs.

Here is an informal argument that shows that there is no predicate on functions that decides where their subject terminates on all inputs. By contradiction, let us assume that there exists f such that

$$f(g) = \begin{cases} \text{true} & \text{if } g \text{ terminates on all inputs,} \\ \text{false} & \text{otherwise.} \end{cases}$$

Now let us define a function h as follows:

$$h(g) = \begin{cases} \text{loops forever} & \text{if } f(g), \\ \text{terminates on all inputs} & \text{otherwise} \end{cases}$$

In other words, h looks at $f(g)$ and does the opposite of what f predicts about g .

Termination is undecidable

Let us consider now $h(h)$:

- If $f(h)$, then, by definition of f , h terminates on all inputs, in particular $h(h)$ terminates. But, by definition of h , since $f(h)$ holds, $h(h)$ does not terminate, yielding a contradiction.
- If $f(h)$ is false, then, by definition of f , h does not terminate on all its inputs. But, by definition of h , since $f(h)$ does not hold, $h(h)$ terminates on all inputs, yielding another contradiction.

A mini functional language

- Rewrite systems are a good mathematical and mental model for understanding functional languages.
- It is time now to learn an actual functional language, so we can run programs.
- We are going to start with a mini language that is actually a subset of OCaml, then grow it step by step.

Growing a language

- The presentation is principled, that is, it relies on a systematic classification of the **syntactic constructs** and their **formal semantics**.
- When we reach a certain complexity, we will drop the formal descriptions.
- Both theoretical and pragmatic, our approach is complementary to those usually found in books.
- Let mini-ML be the name of the subset of OCaml we are going to grow.

Appending a stack

- The notation we introduced for stacks in the context of rewrite systems actually follows the OCaml convention.
- In the context of functional programming, stacks are often called **lists**.
- Here are different ways to express the same list in OCaml:

$$1 :: (2 :: 3 :: (4 :: [])) = 1 :: 2 :: 3 :: 4 :: [] = [1; 2; 3; 4].$$

- Before starting, let us see how a familiar function defined with a rewrite system looks like in OCaml.
- Let us recall the appending of a stack:

$$\begin{aligned} \text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t) \end{aligned}$$

Appending a stack

- In OCaml, the function name is not repeated (once for each rule).
- Instead, it is introduced once by a **binder** called **let**, like so:

```
let cat = function ...
```

- The complete definition in OCaml is:

```
let rec cat = function  
  [], t → t  
| x :: s, t → x :: cat(s, t)
```

- Note the addition of the keyword **rec**: this is meant so the call $\text{cat}(s, t)$ indeed refers to the function being defined.
- With rewrite systems, recursion is not an issue: any call matching a pattern can be rewritten.
- Note also how the two rules are introduced by the keyword **function** and are separated by a vertical line.

Sentences

- An OCaml program is a sequence of sentences.
- A **sentence**, or **global definition**, is defined by the following cases, where e denotes an expression, x is a variable, and **let** and **rec** are keywords:
 - *global definition* **let** $x = e$;;
 - *global recursive definition* **let rec** $x = e$;;
- Global definitions can be optionally followed by two semicolons: these are necessary when inputting the sentences in the toplevel loop (prompted after running the command `ocaml` from a Unix shell).
- Note that the keyword **rec** is necessary to enable recursion. (We will see later how that works precisely.)
- For e to be an expression means that e denotes a part of a sentence which is classified as an expression according to its **syntax**.

Metavariables

- We say that e is a **metavariable** because, being a name, it is a variable, but that variable does not belong to the language being described (the **object language**, here, OCaml) and, instead, exists only in the descriptive language (the **metalanguage**).
- In the following

let $x = e$;;

the metavariable x (mind the italics) denotes an infinite set of OCaml variables and we must not confuse it with the OCaml variable x (no italics). Similarly, the metavariable e denotes an infinity of expressions, as in the following:

let $e = 3$

let $x = e$

let $a = x + e$

Expressions

- An expression e is **recursively defined by** the following **cases**:
 - *variable* $\text{my_var, } x, y, z, \text{ etc.}$
 - *unit or integer constant* $() \text{ or } 0 \text{ or } 1 \text{ or } 2, \text{ etc.}$
 - *function (or abstraction)* **fun** $x \rightarrow e$
 - *call (or application)* $e_1 \ e_2$
 - *arithmetic operator* $(+) \ (-) \ (/) \ (*)$
 - *arithmetic operation* $e_1 + e_2 \text{ or } e_1 - e_2, \text{ etc.}$
 - *parenthesis* (e)
 - *local definition* **let** $x = e_1$ **in** e_2
- Note how the keywords **rec** and **function**, as well as the definition of multiple cases (as seen in the definition of 'cat' earlier) is not found above: this is because we want to focus on other aspects of OCaml first, like higher-order functions.
- Instead we have **fun**, which enables only one case made of a variable.

Examples of expressions

- Note that what we print nicely as “ $\rightarrow$ ” is actually written “`->`” in the source code.
- Here are examples of OCaml sentences making up a program, step by step.
- First, a numerical constant:

let $x = 0$

- Note that constants can be defined by means of a rewrite system as functions taking the empty tuple:

$$x() \rightarrow 0$$

- The next sentence is

let $id = \text{fun } x \rightarrow x$

- This is equivalent to the rewrite system

$$id(x) \rightarrow x$$

Examples of expressions

- Next we have another constant being defined by binding a variable to the value of a call:

let $y = 2$ **in** $\text{id } y$

This is equivalent to the call $\text{id}(2)$.

- Now

let $x = (\text{fun } x \rightarrow \text{fun } y \rightarrow x + y)$ $1\ 2$

is equivalent to

let $x = 1 + 2$

(We will see why later.)

- Finally

let $z = x + 1$

is obvious.

Functions

- Variables must start with a lowercase character.
- The arrow is right-associative, so the expression

$$\mathbf{fun} \ x_1 \rightarrow \mathbf{fun} \ x_2 \rightarrow \dots \rightarrow \mathbf{fun} \ x_n \rightarrow e$$

is equivalent to $\mathbf{fun} \ x_1 \rightarrow (\mathbf{fun} \ x_2 \rightarrow (\dots \rightarrow (\mathbf{fun} \ x_n \rightarrow e)) \dots)$.

- Function calls are left-associative, so

$$e_1 \ e_2 \ e_3 \ \dots \ e_n = (((\dots (e_1 \ e_2) \ e_3) \dots) \ e_n).$$

- Function calls have higher priority than operator calls, for example,

$$f \ 3 + 4 = (f \ 3) + 4 = f \ (3) + 4.$$

- Operator calls have higher syntactic priority than abstractions, for instance,

$$\mathbf{fun} \ x \rightarrow x + y = \mathbf{fun} \ x \rightarrow (x + y).$$

Function calls

- Note the difference between $f\ x\ y$ and $f(x,y)$.
- The former is to be understood as $(f(x))(y)$ which means that there are *two* calls:
 1. First, the call $f(x)$ is evaluated. Let us assume that its value is g .
 2. Second, the call $g(y)$ is evaluated.
- In other words, the first case is equivalent to
let $g = f\ x$ **in** $g\ y$

Abstract syntax

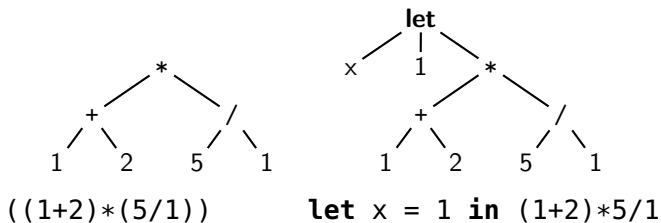
- A graphical representation of programs (here, expressions) as trees proves very handy when studying their semantics (meaning):

Expression	Tree
x	x
$\text{fun } x \rightarrow e$	$\begin{array}{c} \text{fun} \\ / \quad \backslash \\ x \quad e \end{array}$
$e_1 \ e_2$	$\begin{array}{c} \$ \\ / \quad \backslash \\ e_1 \quad e_2 \end{array}$

- Note that we needed some symbol for the root of the application tree (\$). Traditionally, that symbol is (@), but OCaml uses it already for something else.
- When trees represent programs, they are called **abstract syntax trees** (AST).

Abstract syntax

- Intuitively, the method for constructing abstract syntax trees consists in fully parenthesising the expression.
- Each parenthesis corresponds to a subexpression and each subexpression corresponds to a subtree.
- The tree is built from the root, down to the leaves, by traversing subexpressions from the outermost to the innermost, that is, the most embedded ones.
- For instance, consider

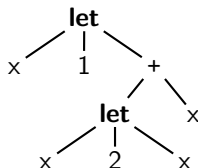


Abstract syntax

- Another example:

let $x = 1$ **in** ((**let** $x = 2$ **in** x) + x)

corresponds to the abstract syntax tree



- This example features two local definitions of 'x': is one the redefinition of the other? Is one hiding the other?
- There are also two uses of 'x': which use corresponds to which definition?

Bindings, environments, scope

- To answer these questions, we must first understand
 - what a **binding** is,
 - what the **scope** of a binding is, and
 - what an **environment** is.
- The expression we called **local definition** earlier:

$$\mathbf{let\ } x = e_1 \mathbf{\ in\ } e_2$$

is evaluated as follows:

1. The expression e_1 is evaluated,
2. assuming it has a value v_1 , then v_1 is **bound** to the variable x ,
3. the expression e_2 is evaluated under the assumption that x is bound to v_1 .

Bindings, environments, scope

- A local definition

let $x = e_1$ **in** e_2

associates the value v_1 of expression e_1 to the variable denoted by x , making up a **binding** noted $x \mapsto v_1$, and then proceeds to evaluate e_2 with this binding available.

- The binding $x \mapsto v_1$ is not usable outside the evaluation of e_2 .
- We say that the **scope** of the definition is made of the subtrees of the abstract syntax tree corresponding to e_2 where the binding of that definition is usable (“visible”).
- A mapping from variables to values is called an **environment**.
- Let us note $\rho_\emptyset$ the empty environment.
- We write

$$x \mapsto v \oplus \rho$$

to mean that the binding $(x \mapsto v)$ was added to the environment ρ .

Bindings, environments, scope

- Consider again the program

```
let x = 0 in
let id = fun x → x in
let y = id x in
let x = (fun x → fun y → x + y) 1 2
in x + 1
```

- The variable x in the expression $x+1$ denotes the value of the expression bound to the variable x in the previous line, not the first.
- Bindings are ordered in the environment **by the order of their definitions**.
- Let us follow the evolution of the environment.

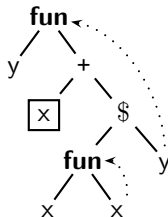
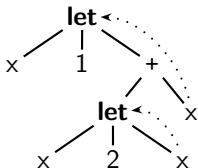
Bindings, environments, scope

1. The environment is initially empty: $\rho_{\emptyset}$;
2. **let** $x = 0$ **in** ...
yields $(x \mapsto 0) \oplus \rho_{\emptyset}$;
3. **let** $id = \text{fun } x \rightarrow x$ **in** ...
yields $(id \mapsto \text{fun } x \rightarrow x) \oplus (x \mapsto 0) \oplus \rho_{\emptyset}$;
4. **let** $y = id\ x$ **in** ...
yields $(y \mapsto 0) \oplus (id \mapsto \text{fun } x \rightarrow x) \oplus (x \mapsto 0) \oplus \rho_{\emptyset}$;
5. **let** $x = \dots$
yields $(\underline{x} \mapsto 3) \oplus (y \mapsto 0) \oplus (id \mapsto \text{fun } x \rightarrow x) \oplus (x \mapsto 0) \oplus \rho_{\emptyset}$.
6. The expression $x+1$ is evaluated in $3 + 1 = 4$ because $x \mapsto 3$ (the last binding) **hides** $x \mapsto 0$ (the first binding).
7. In other words, the first definition of x was **out of scope** in $x+1$.

Free variables

- By definition, a variable is **free** in an expression if it is not bound by a **let** or a **fun**.
- We can understand this concept on a graphic representation of the abstract syntax tree.
- From each variable occurrence, we move up, towards the root.
- If we encounter a **let** that binds that variable (in the left child), we create an oriented edge from the variable to that **let**, and we stop: the variable is bound.
- Otherwise, if we reach the root (no such **let** has been found), then the variable is free.

Free variables



let $x = 1$ **in** ((**let** $x = 2$ **in** x) + x) **fun** $y \rightarrow x + (\text{fun } x \rightarrow x) y$

- A similar situation arises with functions: **fun** is a binder, just like **let**.
- In **fun** $x \rightarrow e$, the variable x (called a **parameter**) may shadow (hide) another variable x defined on the path to the root.
- See above for an example.
- We note $\mathcal{F}(e)$ the set of the variables free in e .

Free variables

- It is easy to formally define $\mathcal{F}$ by case and recursively:

$$\mathcal{F}(x) = \{x\}, \quad \text{where } x \text{ is a variable}$$

$$\mathcal{F}(c) = \emptyset, \quad \text{where } c \text{ is a constant}$$

$$\mathcal{F}(\mathbf{fun} \ x \rightarrow e) = \mathcal{F}(e) \setminus \{x\}$$

$$\mathcal{F}(e_1 \ e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

$$\mathcal{F}(\mathbf{let} \ x = e_1 \ \mathbf{in} \ e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2) \setminus \{x\}$$

$$\mathcal{F}(e_1 + e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

- Now, instead of drawing abstract syntax trees, let us use these equations to compute the free variables of our previous examples.

Free variables

$$\begin{aligned}\mathcal{F}(\text{let } x = 1 \text{ in } (\text{let } x = 2 \text{ in } x) + x) \\&= \mathcal{F}(1) \cup \mathcal{F}((\text{let } x = 2 \text{ in } x) + x) \setminus \{x\} \\&= \emptyset \cup (\mathcal{F}(\text{let } x = 2 \text{ in } x) \cup \mathcal{F}(x)) \setminus \{x\} \\&= (\mathcal{F}(2) \cup \mathcal{F}(x) \setminus \{x\} \cup \mathcal{F}(x)) \setminus \{x\} \\&= \mathcal{F}(x) \setminus \{x\} \\&= \emptyset.\end{aligned}$$

$$\begin{aligned}\mathcal{F}(\text{fun } y \rightarrow x + (\text{fun } x \rightarrow x) y) \\&= \mathcal{F}(x + (\text{fun } x \rightarrow x) y) \setminus \{y\} \\&= (\mathcal{F}(x) \cup \mathcal{F}((\text{fun } x \rightarrow x) y)) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(\text{fun } x \rightarrow x) \cup \mathcal{F}(y)) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(x) \setminus \{x\} \cup \{y\}) \setminus \{y\} \\&= \{x, y\} \setminus \{y\} \\&= \{x\}.\end{aligned}$$

Closed expressions

- A **closed** expression is an expression without free variables.
- That is, an expression e is closed if, and only if, $\mathcal{F}(e) = \emptyset$.
- Only a closed expression can be evaluated (executed).
- By extension, we say that e is **closed in an environment** ρ if $\mathcal{F}(e) \subseteq \text{dom}(\rho)$ (the free variables are bound in ρ).
- That is why the first static analysis performed by compilers consists in determining the variables which are free in expressions: if the program is not closed, it is rejected.
- In the case of **fun** $y \rightarrow x + (\text{fun } x \rightarrow x) y$, the OCaml compiler prints

Unbound value x

and stops. It is useful that this open expression be rejected at compile-time and does not cause a run-time error.